# S6 — Rerank Signal Investigations: What Shipped and What Didn't

---

**Files changed:** `src/rerank.py`, `src/text.py`, `src/state.py` (two new signals + housekeeping); supporting edits in `src/facets.py`, `src/policy.py`, `tools/sweep.py`, `tests/test_components.py`.

**Result:** public set 0.9128 → **0.9159** (200/200 kept), dev split 0.9207 → 0.9233, holdout 0.9010 → 0.9048, adversarial hard set 0.7914 → **0.7944** with no bucket regressing and the worst bucket (`homogeneous_cluster`) up 4.7 MRR points.

This is the record of the rerank-signal investigations. Two signals shipped (negative facet evidence; association-preserving pair spans, plus a word-boundary fix to span matching), one was implemented, measured, and removed (profile-weighted agreement), one was rejected before a line of code was written (budget/price closeness), three fragment-grouping variants were prototyped and rejected, and the no-span early return was challenged and upheld. The negative results are documented with the same care as the positive ones — knowing *why* a plausible signal does not work is what stops it being rebuilt later.

**This doc is the decision log: what was tried and what the measurements said.** Its companion `signal_descriptions.md` is the as-built spec — every shipped signal, exactly how it is computed, and the house method for choosing a weight. Read that one to know what the reranker *does*; read this one to know *why*. Keep them in step: a signal added or a weight changed needs an edit in both.

All numbers were measured on the current tree in one session, so before/after pairs are directly comparable. (The hard-set baseline here is 0.7914, not the 0.8007 quoted for the tail signal in `signal_descriptions.md` and `category_tail_match.md` — the anchor-retrieval merge moved it. Hard-set numbers are only comparable within a single measurement session.)

---

## 1. Shipped: negative facet evidence

### *The gap*

Every existing rerank term is positive evidence: span coverage, facet agreement, category and tail alignment. None of them can tell apart a candidate that *satisfies* "color: grey" from one that merely *doesn't contradict* it — a black-only shirt matches "cotton shirt" spans exactly as well as a grey one and loses nothing for being black. The customer's constraint rules products out; the scorer only ever ruled products in.

### *The signal*

`_facet_conflicts(customer_facets, product_facets, product_text)` counts each stated facet value the candidate contradicts, and the score subtracts `facet_conflict_weight × conflicts`. A conflict requires all three of:

1. **The customer stated a value** for the attribute (via

`extract_query_facets` — material, color, style, use_case, size).

2. **The candidate resolves that attribute too.** Silence is never punished:

a product that mentions no colour at all is not in conflict with "grey". Missing data is not disagreement.

3. **The stated value appears nowhere in the candidate's text** as a

word-bounded substring, aliases included (`grey`/`gray` is the only synonym pair in the facet vocabulary). This guards against `extract()`'s first-match-wins behaviour: a "black/grey reversible" belt extracts `color: black` but still contains "grey", and must not be penalised.

### *The bug the first version had — and what it taught*

The first implementation judged conflicts against `full_text()` (everything the customer ever said). On the adversarial override bucket this *punished the target for obeying the intent override*: after "actually, ignore black — I need grey", the

stale "black" was still in the transcript, still extracted first, and grey-only products (the target among them) were penalised for contradicting it. Measured cost: `generic_override` MRR 0.673 → 0.626.

The fix judges conflicts against `focused_text()` — the currently authoritative turns only, identical to `full_text()` until an override fires. That restored `generic_override` to exactly 0.673 and is now covered by a unit test (`test_override_discards_stale_facet_for_conflict_scoring`). The general lesson is recorded here deliberately: **negative evidence is more override-sensitive than positive evidence.** A stale positive term merely boosts some wrong candidates; a stale negative term actively demotes the right one.

### *Measurements (weight 0.0 → 0.4)*

| split | score off | score on | MRR off | MRR on |
|---|---|---|---|---|
| dev (120) | 0.9207 | **0.9224** | 0.862 | 0.870 |
| holdout (80) | 0.9010 | **0.9021** | 0.811 | 0.815 |
| public full (200) | 0.9128 | **0.9143** | 0.842 | 0.848 |
| hard (96) | 0.7914 | **0.7917** | 0.695 | 0.696 |

Hard set per bucket (MRR): every bucket flat or up, none down — budget_only_signal 0.732 → 0.736, homogeneous_cluster 0.429 → 0.431, the other four unchanged. Public hit rate stays 200/200, which was the disqualifying check: a false conflict that demotes a true target would show up there first.

Weight choice: dev scores 0.9224 at 0.4 and 0.9226 at 0.8 — a plateau, and a penalty term gets the smallest weight on it, so 0.4 shipped. At 0.4, one conflict (–0.4) can reorder saturated ties but cannot overturn a genuine span lead (each matched span is worth at least 1.12).

### *An honest note on the original hypothesis*

This signal was proposed to attack the `homogeneous_cluster` bucket (MRR 0.429), where positive signals saturate. Measurement showed that hypothesis was wrong: bucketmates in a homogeneous cluster share the stated facets by construction, so there is nothing to contradict *within* the cluster and the bucket barely moves (+0.002). The gain comes from the general distribution instead — cross-cluster impostors that match the spans but not the stated colour or material. The signal earned its place on the measured gain, not on the story it was designed around.

---

## 2. Shipped: association-preserving pair spans (+ word-bounded matching)

### *The gap — spotted by reading one transcript*

The customer's turn-2 message in `public_0020`:

> For that, what matters is: color: grey; Solid colors: 100% Cotton; > Heather Grey: 90% Cotton, 10% Polyester; All Other Heathers: 50% Cotton, > 50% Polyester.

`constraint_spans()` splits on `[.;:,\n]`, producing `heather grey`, `90 cotton`, `10 polyester`, … — eight fragments. But the line is a *mapping*: colour-variant → composition. Splitting on colons and commas severs "heather grey" from *its* "90 cotton 10 polyester", so a candidate that pairs the composition differently (an 80/20 heather grey) matches all eight fragments exactly as well as the target. The fragments ask "does this product mention 90% cotton at all?"; the evidence in the message is "does it say that *about heather grey*?"

### *What was tried first — and rejected*

The obvious diagnosis is that eight fragments from one line over-weight that line, so three de-weighting variants were prototyped: per-utterance groups scored as sum over sqrt(k) (public 0.9139), mean (0.9134), and best-single-span (0.9025, –1.2 points). **All flat or worse.** The fragment gradient — target matches 8/8, impostor matches 5/8 — is load-bearing and must not be compressed. The fix is not to weaken the fragments but to *add* the association they lost.

### *The signal*

`pair_spans()` (`src/text.py`) splits only on sentence separators (`.;\n` and `" - "`), keeping colon/comma-joined content together, stripping the leading simulator framing, minimum 3 words. `query_pair_spans()` (`src/state.py`) applies the same turn-1/declined exclusions as `query_spans()` and drops anything already emitted as a fragment. The reranker adds `pair_weight (0.8) × matched pairs`, flat 1.0 per pair — the pair's evidential value is the intact association, not its length. Catalog copy repeats these blocks verbatim, so the joined form is still an exact substring of the target: df("heather grey 90 cotton 10 polyester") = 511 products vs 612 for "heather grey" alone.

Alongside it, one latent bug fixed: coverage tested `span in text` unanchored, so `"90 cotton"` also matched `"190 cotton"` (1–4 catalog products per numeric span). Both fragment and pair matching are now word-bounded — the product text is token-joined, so padding with single spaces anchors every span at token edges.

### *Measurements*

| step | public | public MRR | hard | hard MRR |
|---|---|---|---|---|
| baseline (conflict shipped) | 0.9143 | 0.848 | 0.7917 | 0.696 |
| + word-bounded matching | 0.9149 | 0.850 | 0.7917 | 0.696 |
| + pair spans (0.8) | **0.9159** | **0.851** | **0.7944** | **0.705** |

Hard set per bucket: the entire gain sits in `homogeneous_cluster` (MRR 0.431 → **0.478**) and `generic_override` (0.673 → 0.680); every other bucket is exactly unchanged, and no hit is lost anywhere. This is the bucket the conflict signal could not move — bucketmates in a homogeneous cluster share every *fragment* by construction, but they do not all pair the compositions the same way, so the intact association is the discriminator that survives saturation. `public_0020` itself, the session that exposed the gap, goes from rank 4 to **rank 1**.

Weight: 0.4–1.5 score identically on dev (0.9233) and holdout (+0.0008); 0.8 sits mid-plateau per house convention.

---

## 3. Implemented, measured, removed: profile-weighted facet agreement

`user_profile.preference_tags` (e.g. `["material", "fit"]`) is unused in the scored path. The idea: give extra credit when a facet agreement lands on an attribute the profile says this customer cares about — a personalisation prior, additive-only, mapped through the shared `TAG_HINTS` vocabulary.

| config | dev | holdout |
|---|---|---|
| baseline (conflict shipped) | 0.9224 | 0.9021 |
| profile_weight 0.1 | 0.9237 | 0.9013 |
| profile_weight 0.2 | 0.9217 | 0.9005 |
| profile_weight 0.4 | 0.9217 | — |

The dev gain at 0.1 does not reproduce on holdout (–0.0008, and monotonically worse as the weight grows). The shape is a knife-edge, not a plateau — the signature of fitting noise. Removed per the repo convention (dead options are deleted, not kept at weight 0; the pool-local-rarity precedent).

Why it fails is worth keeping: **the profile is redundant with the transcript.** The simulated customer already discloses their profiled preferences verbatim during the session — a customer whose profile says "material" *says the material out loud* by turn 2 or 3, and the facet-agreement term already scores it. Re-weighting that same agreement adds variance without adding information. A profile signal would only pay off where the profile carries information the conversation lacks, which this evaluator's design never produces.

One durable side effect kept: `TAG_HINTS` moved from `InfoGainPolicy` to `src/facets.py` (module level) with identity entries added, so a literal tag like `"material"` now maps to the `material` attribute. `InfoGainPolicy` aliases it unchanged; profile tags that name an attribute directly previously did nothing in its answerability prior.

### *3b. Same verdict, second try: rating-disposition modulated popularity*

A different profile field, a fresh hypothesis: `average_prior_rating` / `rating_style` are read by nothing and are genuinely orthogonal to the transcript (a customer never says "I rate harshly"). Hypothesis — a generous rater buys

mainstream, well-reviewed products; a critical one shops away from the bestseller shelf. Added term: `popularity_profile_weight x rating_disposition x _popularity(product)`, with `rating_disposition = clip(average_prior_rating - 4.0, -1, +1)` (the public groups avg 5 / 4 / <=3 map to +1 / 0 / -1).

| weight | dev | holdout |
|---|---|---|
| 0.00 | 0.9233 | 0.9048 |
| 0.02 | 0.9235 | 0.9048 |
| 0.05 | 0.9254 | 0.9033 |
| 0.10 | 0.9285 | 0.9014 |
| 0.20 | 0.9293 | 0.9036 |

Dev climbs monotonically (+0.006); holdout is best at 0 and down ~0.003 across the range — the identical knife-edge. Full public "+0.0018" at 0.10 is the dev gain diluted by the holdout loss; the hard set "+0.005" is not independent (its profiles are synthesised with a rating correlation by `tools/hard_cases.py`). Code reverted. Two independent profile attempts have now failed the same way: **the `user_profile` carries no non-redundant rerank signal on this evaluator.** The disposition-to-popularity correlation is real in the 120 dev sessions and noise in the 80 holdout ones.

---

## 4. Rejected before implementation: budget/price closeness

The idea — long recorded in `docs/team/hard_cases.md` as "budget is triply dead" — was to parse the customer's "budget around $X" and score candidates by price closeness, since the disclosed figure is the target's own price.

Measurement against the evaluator's own `intent_card()` killed it before any code was written:

| population | cards containing a budget constraint |
|---|---|
| public set (200 sessions) | **0** |
| hard set (96 sessions) | 4 — all rendered `budget around $`– (no number) |
| full catalog (50,000 intent cards) | 224 (**0.45%**), of which 110 parseable (**0.22%**) |

The root cause is structural: `intent_card()` appends the budget line *after* all feature/detail candidates and keeps only the first four, so a budget surfaces only for near-empty "degenerate card" products. The four hard-set sessions that do carry one are exactly the four `never_retrieved` misses — the target is not in the pool, so no rerank term of any kind can reach them. And their dollar figure renders as "—" anyway.

Expected effect on a private 800 drawn the same way: ~2 sessions, likely zero. The signal was not built. If a future evaluator version emits budgets routinely, the design is on file: parse the last-mentioned dollar amount, score 1.0 within ±10% with linear decay to ±50% (never exact-match — robust to rounding), weight ~0.6.

---

## 5. Challenged and upheld: the no-span early return

`rerank()` returns the retrieval order untouched when `state.query_spans()` is empty, and `query_spans()` excludes turn 1. The objection is fair on its face: *"no spans" is not "no information"* — the opening always names a category, and in buying sessions it also states a hard requirement. Three situations were separated and measured.

### *How much is even at stake*

The early return fires on a turn that **actually shows a list** in only 20 of 595 public turns (3.4%) and 18 of 372 hard turns (4.8%). Most no-span turns are turns 1-2, where the list is withheld anyway. That ceiling caps any possible gain before a single variant is written.

### *Case: the buying opening's hard requirement — a non-issue*

`intent_card()` inserts the matched material at position 0, so the `hard_constraints[0]` that the opening quotes is almost always a **single word**: "cotton", "leather", "silk", "Material:alloy". `constraint_spans()` requires `min_words=2`,

so there is no span to extract even if turn 1 were included, and nothing is being discarded: that requirement already reaches ranking twice, as a BM25 query term and as a `material` facet via `extract_query_facets`. What turn 1 *would* add is mostly framing noise ("i m looking for jewelry necklaces", "but i m still exploring"). The one real exception is `intent_override`, whose opening carries genuine product copy ("stainless steel band", "buckle closure") — which is precisely the value the customer later discards.

### *Case: a later turn with no preference — already correct*

Declined turns are held out of spans *and* of `full_text()`, while earlier real constraints remain. Prior constraints therefore continue to drive ranking after a decline; the early return fires only when nothing was **ever** disclosed.

### *Case: vague opening with only a category — measured*

Two variants, and their combination:

***A*** *— when there are no spans, rescore with category signals only (retrieval + popularity + category + tail, no span/pair/facet/conflict terms).* **B** — include turn-1 spans minus the leading framing fragment.

| variant | dev | holdout | public | hard |
|---|---|---|---|---|
| baseline (shipped) | 0.9233 | 0.9048 | 0.9159 | 0.7944 |
| A only | 0.9233 | 0.9070 | 0.9168 | 0.7938 |
| B only | 0.9237 | 0.9048 | 0.9162 | 0.7940 |
| A + B | 0.9237 | 0.9070 | 0.9170 | 0.7960 |

A alone is dev-flat; B alone is holdout-flat; only the combination moves both, and the interaction has no mechanism behind it. The session counts show what the combination actually is: across 296 sessions it moves **4 better / 1 worse on public** and **5 better / 2 worse on hard**. The holdout +0.0022 is arithmetically one session going from rank 3 to rank 1 — an order of magnitude below the ~0.02 noise floor `tools/sweep.py` documents for that split. Per bucket, A+B trades `degenerate_card` (MRR 0.527 -> 0.555) against `generic_override` (0.680 -> **0.664**).

The refinement the bucket split suggests — keep turn-1 spans but drop them once an override fires — was tested and made that bucket **worse** (0.623), not better. That independently re-confirms the reasoning already recorded in `apply_override` (src/state.py): in this evaluator the discarded preference is still derived from the target product, so erasing it destroys usable signal.

**Not shipped.** With hit@10 already at 1.0, a change that moves 5 of 296 sessions on an unexplained interaction, while regressing a bucket, is the overfitting signature this document exists to catch. The early return and the turn-1 exclusion stand.

---

## 6. Housekeeping shipped alongside

*Deleted the dead commented-out block in* `_facet_agreement` *(the pre-lowercase* `extract()` *call kept as a* `'''…'''` *string).* `extract_query_facets(state.full_text())` was recomputed for **every candidate** (300× per turn); it is now computed once per `rerank()` call and passed in. Verified bit-identical before and after. * The scoring formula in `docs/team/ideas.md` was three signals out of date (still showed span + bm25 + popularity over depth 200); now matches `RerankConfig`.

## 7. Reproduction

```
python3 -m unittest discover -s tests            # 57 tests
python3 -m evaluator.local_evaluator             # 0.9159, hit 1.0
python3 -m evaluator.local_evaluator \
    --dataset data/hard_set.jsonl                # 0.7944
python3 tools/sweep.py --split dev \
    --configs conflict00,conflict04,pair00,pair08   # the ablation rows
python3 tools/observe.py --only public_0020      # the motivating session, now rank 1
```

New unit tests (`tests/test_components.py`): conflict demotes a contradicting candidate; a multi-value product containing the stated value is not punished; silence about a facet is not a conflict; an override discards the stale value

for conflict scoring; pair spans keep key:value associations and strip leading filler; an intact association outranks recombined fragments; span matching is word-bounded; each new weight at 0.0 reproduces the previous ranking exactly.